

CURRICULUM INTELLIGENCE CHALLENGE

SECTION 1: My Current Degree

I am currently pursuing B.Tech in Computer Science and Engineering (CSE) at JNTUH College of Engineering Sultanpur (JNTUHCES), under Jawaharlal Nehru Technological University Hyderabad (JNTUH).

I am studying under the R22 Regulations. For this report, I am using the current R22 CSE curriculum as the baseline. I will analyse what is being taught in the curriculum today and what may need to change or improve for CSE students who will graduate in 2030.

1.1 Degree Details

Particular	Details
Degree	B.Tech – Computer Science and Engineering
Branch	Computer Science and Engineering (CSE)
College	JNTUH College of Engineering Sultanpur (JNTUHCES)
University	Jawaharlal Nehru Technological University Hyderabad (JNTUH)
Regulation	R22
Programme Duration	4 Years / 8 Semesters
Total Credits	160

1.2 Syllabus Document Used

For this report, I used the official JNTUH syllabus document.

Exact Document Title:

R22 B.Tech. CSE Syllabus – Course Structure & Syllabus (R22 Regulations)

University: Jawaharlal Nehru Technological University Hyderabad (JNTUH)

Applicable From: AY 2022–23 Batch

Official Source: JNTUH

Official Syllabus URL:

<https://jntuh.ac.in/uploads/academics/R22B.Tech.CSECourseStructureSyllabus.pdf>

Date Downloaded: 12/09/2026

1.3 What I Am Using This Curriculum For

I am using the R22 curriculum as the starting point for the rest of this report.

First, I map all 8 semesters, including the courses, laboratories, projects, electives and credits. Then, I look at which parts of the curriculum are still useful, which parts may need updating, and what may be missing when we consider the skills expected from a CSE graduate in 2030.

I am not assuming that an older subject is automatically outdated. Some subjects provide fundamental Computer Science knowledge that can remain useful for many years. The aim of this report is to separate these strong foundation subjects from the areas that may need more current tools, practical work and industry-oriented learning.

The main idea is that the Computer Science foundation should not be removed. Instead, the curriculum can be rebalanced so that students have enough time to learn the fundamentals and also apply them to modern and future technologies.

SECTION 2: What I Am Being Taught

Before deciding what should change in the curriculum, I first mapped what my current R22 B.Tech CSE programme actually contains.

The programme is divided into 8 semesters, with 20 credits in each semester, giving a total of 160 credits. The curriculum includes mathematics and basic engineering subjects, core Computer Science subjects, laboratories, professional electives, open electives, projects and an internship/skill-development component.

2.1 First Year – I Semester

Course Code	Course Title	Credits
MA101BS	Matrices and Calculus	4
CH102BS	Engineering Chemistry	4
CS103ES	Programming for Problem Solving	3
EE104ES	Basic Electrical Engineering	2
ME105ES	Computer Aided Engineering Graphics	3
CS106ES	Elements of Computer Science & Engineering	1
CH107BS	Engineering Chemistry Laboratory	1
CS108ES	Programming for Problem Solving Laboratory	1
EE109ES	Basic Electrical Engineering Laboratory	1
Total		20

2.2 First Year – II Semester

Course Code	Course Title	Credits
MA201BS	Ordinary Differential Equations and Vector Calculus	4
PH202BS	Applied Physics	4
ME203ES	Engineering Workshop	2.5
EN204HS	English for Skill Enhancement	2
EC205ES	Electronic Devices and Circuits	2
CS206ES	Python Programming Laboratory	2
PH207BS	Applied Physics Laboratory	1.5
EN208HS	English Language and Communication Skills Laboratory	1
CS209ES	IT Workshop	1
MC210	Environmental Science	0
Total		20

2.3 Second Year – I Semester

Course Code	Course Title	Credits
CS301PC	Digital Electronics	3
CS303PC	Computer Oriented Statistical Methods	4
CS304PC	Computer Organization and Architecture	3
CS305PC	Object Oriented Programming through Java	3
CS306PC	Data Structures Lab	1.5
CS307PC	Object Oriented Programming through Java Lab	1.5
CS308PC	Data Visualization – R Programming / Power BI	1
MC309	Gender Sensitization Lab	0
Total		20

2.4 Second Year – II Semester

Course Code	Course Title	Credits
CS401PC	Discrete Mathematics	3
SM402MS	Business Economics & Financial Analysis	3
CS403PC	Operating Systems	3
CS404PC	Database Management Systems	3
CS405PC	Software Engineering	3
CS406PC	Operating Systems Lab	1
CS407PC	Database Management Systems Lab	1
CS408PC	Real-time Research Project / Societal Related Project	2
CS409PC	Node JS / React JS / Django	1
MC410	Constitution of India	0
Total		20

2.5 Third Year – I Semester

Course Code	Course Title	Credits
CS501PC	Design and Analysis of Algorithms	4
CS502PC	Computer Networks	3
CS503PC	DevOps	3
—	Professional Elective-I	3
—	Professional Elective-II	3
CS504PC	Computer Networks Lab	1
CS505PC	DevOps Lab	1

Course Code	Course Title	Credits
EN508HS	Advanced English Communication Skills Lab	1
CS506PC	UI Design – Flutter	1
MC510	Intellectual Property Rights	0
Total		20

2.6 Third Year – II Semester

Course Code	Course Title	Credits
CS601PC	Machine Learning	3
CS602PC	Formal Languages and Automata Theory	3
CS603PC	Artificial Intelligence	3
—	Professional Elective-III	3
—	Open Elective-I	3
CS604PC	Machine Learning Lab	1
CS605PC	Artificial Intelligence Laboratory	1
—	Professional Elective-III Lab	1
CS606PC	Industrial Oriented Mini Project / Internship / Skill Development Course (Big Data – Spark)	2
MC609	Environmental Science	0
Total		20

2.7 Fourth Year – I Semester

Course Code	Course Title	Credits
CS701PC	Cryptography and Network Security	3
CS702PC	Compiler Design	3
—	Professional Elective-IV	3
—	Professional Elective-V	3
—	Open Elective-II	3
CS703PC	Cryptography and Network Security Lab	1
CS704PC	Compiler Design Lab	1
CS705PC	Project Stage – I	3
Total		20

2.8 Fourth Year – II Semester

Course Code	Course Title	Credits
CS801PC	Organizational Behavior	3
—	Professional Elective-VI	3
—	Open Elective-III	3
—	Project Stage – II including Seminar	11
Total		20

2.9 Professional Electives

Professional Elective-I

Course Code	Elective
CS511PE	Quantum Computing
CS512PE	Advanced Computer Architecture
CS513PE	Data Analytics
CS514PE	Image Processing
CS515PE	Principles of Programming Languages

Professional Elective-II

Course Code	Elective
CS521PE	Computer Graphics
CS522PE	Embedded Systems
CS523PE	Information Retrieval Systems
CS524PE	Distributed Databases
CS525PE	Natural Language Processing

Professional Elective-III

Course Code	Elective
CS631PE	Full Stack Development
CS632PE	Internet of Things
CS633PE	Scripting Languages
CS634PE	Mobile Application Development
CS635PE	Software Testing Methodologies

Professional Elective-IV

Course Code	Elective
CS741PE	Graph Theory
CS742PE	Cyber Security
CS743PE	Soft Computing
CS744PE	Cloud Computing
CS745PE	Ad hoc & Sensor Networks

Professional Elective-V

Course Code	Elective
CS751PE	Advanced Algorithms
CS752PE	Agile Methodology
CS753PE	Robotic Process Automation
CS754PE	Blockchain Technology
CS755PE	Software Process & Project Management

Professional Elective-VI

Course Code	Elective
CS861PE	Computational Complexity
CS862PE	Distributed Systems
CS863PE	Deep Learning
CS864PE	Human Computer Interaction
CS865PE	Cyber Forensics

2.10 Open Electives

Open Elective-I

Course Code	Elective
CS611OE	Data Structures
CS612OE	Database Management Systems

Open Elective-II

Course Code	Elective
CS721OE	Operating Systems
CS722OE	Software Engineering

Open Elective-III

Course Code	Elective
CS831OE	Algorithms Design and Analysis
CS832OE	Introduction to Computer Networks

2.11 My Initial Understanding

This mapping shows that my current CSE degree already has a strong Computer Science foundation. It covers programming, data structures, algorithms, databases, operating systems, networks, software engineering and theory, along with AI, ML, DevOps and application development.

However, simply having a subject in the syllabus does not automatically mean that the subject is being taught with the tools and practices currently used in industry.

Therefore, in the next sections of this report, I will compare this curriculum with the 2026 technology and industry situation and then decide what should be **KEEP, UPDATE, REDUCE or ADD** for students graduating in 2030.

SECTION 3: Foundation vs Current Relevance – Student Enquiry with Lecturer

To understand whether the R22 CSE curriculum can prepare students for the requirements of 2030, we discussed the syllabus with a CSE lecturer. The discussion focused on what should be **kept, updated, reduced, integrated, reallocated and made more practical**.

The lecturer's overall view was that the **Computer Science foundation should remain**, because subjects such as Programming, Data Structures, Algorithms, DBMS, Operating Systems and Networks continue to be important. However, the way students **apply these fundamentals** needs to change with technology.

Instead of continuously adding new subjects, the lecturer suggested **rebalancing existing curriculum time** towards practical development, emerging technologies, projects, employability and responsible use of AI.

Lecturer's Suggested Approach:

Decision	Meaning
KEEP	Retain strong and long-term CSE fundamentals.
UPDATE	Add current tools, applications and practical activities.
REDUCE / INTEGRATE	Reduce lower-priority standalone content and connect it with other activities.
REALLOCATE	Use available curriculum time for emerging and practical skills.
APPLY	Connect theoretical learning with projects and real-world problems.
COMMUNICATE	Develop the ability to explain technical work and work effectively with others.

Key message from the lecturer: “Do not replace the Computer Science foundation; create more opportunities to apply it to modern technologies.”

3.1 Programming for Problem Solving – KEEP

The lecturer considered programming a permanent CSE skill. Even when AI tools can generate code, students still need to understand the logic behind the solution and be able to debug, test, verify and explain the code.

Focus: Problem → Algorithm → Code → Test → Debug → Explain

Decision: KEEP — Programming remains fundamental for software development and code verification.

3.2 Data Structures – KEEP

The lecturer emphasized that students should understand not only how to implement data structures, but also **which data structure to select and why**. Understanding efficiency, complexity and trade-offs remains important even when AI tools suggest solutions.

Focus: Problem → Selection → Complexity → Implementation → Testing

Decision: KEEP — Data Structures is a long-term Computer Science competency.

3.3 Object-Oriented Programming – UPDATE

The lecturer felt that OOP concepts remain highly relevant, but students should connect them with actual software-development practices. Practical activities can include Git, APIs, testing, debugging and code review.

Focus: OOP → Git → API → Testing → Code Review

Decision: UPDATE — Strengthen the practical use of OOP in software development.

3.4 Database Management Systems – UPDATE

The lecturer suggested connecting SQL, normalization, transactions and database concepts with complete applications. Students should also gain exposure to database security, optimization, cloud databases and checking AI-generated SQL before using it.

Focus: Frontend → API → Database → Response

Decision: UPDATE — Move from mainly database theory towards application-based learning.

3.5 Operating Systems – KEEP

The lecturer considered processes, memory management, scheduling and file systems to be stable foundations of Computer Science. However, their connection with modern servers, virtualization, containers and cloud computing should be demonstrated through practical examples.

Focus: OS → Servers → Virtualization → Containers → Cloud

Decision: KEEP — Fundamental operating-system knowledge remains relevant.

3.6 Computer Networks – UPDATE

The lecturer recommended retaining networking fundamentals while making their connection with modern applications clearer. Students should understand DNS, HTTPS, APIs, client-server communication, cloud networking, troubleshooting and basic network security.

Focus: Application → DNS → HTTPS → Server → Response

Decision: UPDATE — Strengthen modern application and cloud-related networking skills.

3.7 Software Engineering – UPDATE

The lecturer suggested making Software Engineering more practical instead of focusing mainly on models, definitions and theory. Students should experience a realistic software-development workflow involving requirements, version control, coding, reviews, testing and deployment.

Focus: Requirement → Git → Code → Review → Test → Deploy

Decision: UPDATE — Emphasize teamwork, version control, testing, deployment and AI-assisted development.

3.8 Design and Analysis of Algorithms – KEEP

According to the lecturer, AI tools may suggest algorithmic solutions, but students still need to judge whether those solutions are correct, efficient and scalable.

Students should therefore continue developing algorithmic thinking, complexity analysis and problem-solving ability.

Decision: KEEP — Algorithmic thinking is independent of any particular programming language or technology.

3.9 DevOps – UPDATE

Since DevOps is already included in the R22 curriculum, the lecturer suggested increasing **hands-on implementation** rather than introducing another separate theory subject.

Focus: Code → Git → Build → Test → Container → Deploy → Monitor

Decision: UPDATE — Increase project-based DevOps practice.

3.10 Machine Learning – UPDATE

The lecturer suggested that ML should be taught as a complete lifecycle rather than focusing mainly on model training.

Focus: Data → Preprocess → Model → Evaluate → Deploy → Monitor

Students should also understand data quality, bias, model limitations and responsible use of ML systems.

Decision: UPDATE — Teach Machine Learning as a complete practical system.

3.11 Artificial Intelligence – UPDATE

The lecturer suggested retaining classical AI concepts such as search, A*, agents and logic because they provide important foundations. At the same time, students should be introduced to modern areas such as Generative AI, LLMs, AI-assisted development, AI agents, output evaluation and responsible AI.

Decision: UPDATE — Combine classical AI foundations with current AI developments.

3.12 Formal Languages and Automata Theory – KEEP

The lecturer considered FLAT to be a stable theoretical foundation. Its relevance can be made clearer by connecting it with compilers, programming languages and the theory of computation.

Decision: KEEP — Important theoretical Computer Science knowledge should be retained.

3.13 Human + AI Professional Readiness – INTEGRATE / STRENGTHEN

The lecturer highlighted that employability should go beyond traditional HR preparation. By 2030, graduates may encounter human interviews, AI-assisted assessments and mixed recruitment processes.

Students should therefore develop:

- Technical and project explanation
- HR and behavioural skills
- Group discussion
- Presentation skills
- Resume communication
- Problem explanation

- Workplace communication
- Teamwork
- Responsible AI use and verification

Development Cycle:

Learn → Practice → Communicate → Feedback → Improve → Interview

Suggested Integration:

Instead of creating another large theory subject, professional readiness can be developed through existing activities:

Communication Labs → Projects → Internships → Placement Training → Industry Interaction

Year	Focus
1st Year	Communication and presentation
2nd Year	Teamwork and technical explanation
3rd Year	Group discussions, projects and workplace communication
4th Year	Technical and HR interviews and professional readiness

Decision: INTEGRATE / STRENGTHEN — Employability requires technical knowledge along with communication, adaptability, teamwork and responsible human-AI collaboration.

3.14 Environmental Science – REDUCE / INTEGRATE

The lecturer suggested retaining environmental awareness but connecting it with CSE-related areas such as green computing, energy-efficient systems, sustainable data centres and e-waste.

R22 lists Environmental Science as a **0-credit course**.

Decision: REDUCE / INTEGRATE — Retain environmental awareness without allocating significant standalone curriculum time.

3.15 Intellectual Property Rights – REDUCE / INTEGRATE

The lecturer suggested teaching essential IPR concepts such as copyright, patents, software licensing, open-source licensing and ownership through projects and innovation activities.

R22 lists IPR as a **0-credit course**.

Decision: REDUCE / INTEGRATE — Make IPR more practical and connected to student projects.

3.16 Constitution of India – REDUCE / INTEGRATE

The lecturer considered civic awareness important but suggested that it could be covered through shorter interdisciplinary or professional activities rather than extensive standalone treatment.

R22 lists Constitution of India as a **0-credit course**.

Decision: REDUCE / INTEGRATE — Preserve civic awareness while reducing standalone academic burden.

3.17 Gender Sensitization – REDUCE / INTEGRATE

The lecturer suggested integrating inclusion, respectful communication and workplace diversity into team projects, professional ethics, communication and workplace-readiness activities.

Decision: REDUCE / INTEGRATE — Retain the intended learning outcomes through broader professional-development activities.

3.18 Engineering Graphics / Drawing – REDUCE

The lecturer noted that basic visualization skills are useful, but extensive traditional drawing has limited direct relevance to many CSE career paths.

R22 allocates **3 credits to Computer Aided Engineering Graphics**.

Decision: REDUCE — Retain useful visualization concepts while creating more space for core computing and practical CSE skills.

3.19 First-Year Engineering Subjects – REDUCE DEPTH, NOT REMOVE

The lecturer did not recommend completely removing Chemistry, Physics, Electrical, Electronics or Workshop. Instead, the depth of these subjects can be reviewed and their connection with CSE applications can be highlighted.

Subject	Possible CSE Connection
Physics	Sensors, semiconductors and hardware
Electrical / Electronics	Digital systems and embedded systems
Chemistry	Batteries, materials and e-waste
Workshop	Prototyping and engineering awareness

Decision: REDUCE DEPTH, NOT REMOVE — Maintain multidisciplinary engineering awareness without excessive theoretical burden.

3.20 Mathematics and Statistics – REDUCE SELECTIVELY

The lecturer emphasized that mathematics remains essential for algorithms, AI, ML, data science and cryptography. Therefore, mathematics should not be removed. Instead, repetitive portions can be reduced and more emphasis can be placed on CSE-relevant applications.

Focus: Probability → Statistics → Optimization → Algorithms → ML → Cryptography

Decision: REDUCE SELECTIVELY — Reduce repetition, not mathematical foundations.

3.21 Repetitive and Exam-Oriented Content – REDUCE

The lecturer identified repetitive memorization, similar programming exercises, highly prescriptive laboratory procedures and tool-based procedures without conceptual understanding as areas that could be reduced.

The available time could instead support:

Problem Solving → Debugging → Projects → Testing → Code Review → Deployment → Communication

Decision: REDUCE — Shift the learning pattern from exam completion towards practical competency.

3.22 Reallocation of Reduced Curriculum Space

The lecturer's main suggestion was that reduced curriculum time should **not simply be replaced with more theory**. Instead, the available time should create opportunities for students to build and demonstrate complete systems.

Proposed Practical Flow:

**Problem → Design → Code → Data → API → Testing → Security → Git →
Deployment → Documentation → Presentation**

Priority Areas

Area	Practical Focus
AI	GenAI, LLMs, evaluation and limitations
Cloud	Deployment and cloud architecture
Cybersecurity	Secure coding and security testing
DevOps	Git, CI/CD and containers
Data Engineering	Data processing and pipelines
Software Development	APIs, testing, debugging and code review
AI-Assisted Development	Generate → Inspect → Test → Improve
Professional Skills	Communication, teamwork and documentation
Human-AI Readiness	Interview, explanation and responsible AI use

Overall Finding from the Lecturer Discussion:

From our discussion, we understood that the R22 CSE curriculum **does not need to be completely replaced**. Its fundamental Computer Science knowledge should remain, while its practical application, exposure to emerging technologies and employability preparation should be strengthened.

Current Pattern:

Learn → Memorize → Exam → Complete Subject

Suggested Pattern:

Learn → Solve → Build → Use Tools → Test → Verify → Deploy → Communicate

The important change is therefore not simply **adding more subjects**, but making better use of the existing curriculum time.

SECTION 4: The 2026 Reality – What Has Changed Since the Curriculum Was Written?

The R22 CSE curriculum was designed for the 2022–23 batch. The core Computer Science subjects included in the curriculum are still useful and form the foundation of a CSE degree. However, the technology environment in which these subjects are applied has changed significantly.

From my review, I identified eight major changes that directly affect the relevance of the R22 CSE curriculum. These changes do not mean that the existing subjects should be removed. Instead, they show where the practical side of the curriculum needs to catch up with current technology and industry practices.

The main changes identified are:

1. Generative AI has become part of software development.
2. LLMs and AI agents have created a new layer of AI development.
3. AI and Big Data skills have become major industry requirements.
4. Cloud-native development has moved into mainstream production.
5. Software development has become more automated and collaborative.
6. Cybersecurity has become a broader requirement.
7. Python has become more important with AI and data.
8. The skills expected from graduates are changing quickly.

4.1 Generative AI Has Become Part of Software Development

AI is no longer only a topic studied in an Artificial Intelligence subject. Developers are increasingly using AI tools for coding, debugging, explaining code and solving programming problems. The 2025 Stack Overflow Developer Survey reported that 84% of respondents were using or planning to use AI tools, while 46% distrusted the accuracy of AI output.

This means that programming is changing from a process mainly based on:

Write → Compile → Debug

towards a workflow such as:

Understand → Generate/Write → Test → Verify → Debug

Affected course:

CS103ES – Programming for Problem Solving

What should change:

Programming fundamentals should remain unchanged, but practical exercises can include AI-generated code review, testing, debugging and verification.

Why it matters:

Students need to learn how to use AI as a development aid without blindly accepting its output. They should still be able to understand the logic of a program and identify incorrect or unsafe code.

Source: [Stack Overflow Developer Survey 2025](#)

Accessed: September 2026

4.2 LLMs and AI Agents Have Created a New Layer of AI Development

The R22 Artificial Intelligence syllabus provides important foundations such as intelligent agents, search, knowledge representation and planning. These concepts remain relevant. However, the current AI environment has expanded significantly with large language models (LLMs), AI assistants and AI agents.

The 2025 Stack Overflow Developer Survey also separately tracks AI agents and AI-assisted development, showing the growing presence of these technologies in software development.

Affected course:

CS603PC – Artificial Intelligence

What should change:

Classical AI foundations should be retained, while the course can introduce modern AI applications, basic LLM concepts, AI-assisted development, evaluation of AI output, limitations and responsible use of AI agents.

Why it matters:

Students should understand the concepts behind modern AI systems rather than learning only how to use one particular AI tool. This makes the knowledge more useful even when specific tools and models change.

Source: [Stack Overflow Developer Survey 2025](#)

Accessed: September 2026

4.3 AI and Big Data Skills Have Become Major Industry Requirements

AI and data have become important across many areas of software development. The World Economic Forum's *Future of Jobs Report 2025* identifies **AI and big data** as the fastest-growing skill area, followed by **networks and cybersecurity** and **technological literacy**. AI

and Machine Learning Specialists and Software and Applications Developers are also identified among the fastest-growing job roles.

Affected courses:

CS601PC – Machine Learning

CS603PC – Artificial Intelligence

What should change:

ML and AI concepts should be connected more strongly with practical data processing, preprocessing, model evaluation and real-world applications.

A useful practical flow would be:

Data → Preprocessing → Model → Evaluation → Application/Deployment

Why it matters:

Knowing an algorithm alone is not enough. Students should understand how an AI/ML system works as a complete solution, from data preparation to evaluation and practical use.

Source: [World Economic Forum, Future of Jobs Report 2025](#)

Accessed: September 2026

4.4 Cloud-Native Development Has Moved into Mainstream Production

Cloud development has moved beyond simply hosting an application on a remote server. Modern software infrastructure increasingly involves containers, orchestration, automated deployment and monitoring.

The CNCF Annual Cloud Native Survey published in January 2026 reported that **82% of container users run Kubernetes in production** and that cloud-native adoption had reached **98% among surveyed organizations**.

Affected courses:

CS503PC – DevOps

CS505PC – DevOps Lab

What should change:

DevOps practical work should include containers, basic orchestration concepts, deployment, monitoring and cloud-native development workflows.

For example:

Code → Container → Deploy → Monitor

Why it matters:

Students should understand what happens after writing an application. They should get practical exposure to how software is packaged, deployed and monitored in a modern environment.

Source: [CNCF, Annual Cloud Native Survey 2026](#)

Accessed: September 2026

4.5 Software Development Has Become More Automated and Collaborative

Software development is no longer mainly an individual activity where a student writes code and submits the final program. Modern development commonly involves Git-based collaboration, branches, pull requests, automated testing, continuous integration and continuous deployment.

The 2026 CNCF survey reports high CI/CD adoption among mature organizations, showing that automated development and delivery practices have become an important part of modern software workflows.

Affected courses:

CS405PC – Software Engineering

CS503PC – DevOps

What should change:

Students should practise a development workflow such as:

Git → Branch → Pull Request → Test → Build → Deploy

instead of learning the software development process mainly through theoretical models.

Why it matters:

This gives students practical experience that is closer to collaborative software development. It also helps connect Software Engineering concepts with the DevOps practices used in real projects.

Source: [Cloud Native Computing Foundation, Annual Cloud Native Survey 2026](#)

Accessed: September 2026

4.6 Cybersecurity Has Become a Broader Requirement

Security is no longer limited to a single cybersecurity or cryptography subject. Modern applications connect databases, APIs, networks, cloud services and external systems. As a result, security considerations are required at different stages of software development.

The World Economic Forum identifies **networks and cybersecurity** as one of the fastest-growing skill areas for the 2025–2030 period.

Affected courses:

CS502PC – Computer Networks

CS404PC – Database Management Systems
CS701PC – Cryptography and Network Security

What should change:

Security examples can be introduced earlier through topics such as authentication, secure APIs, database security, access control and network security.

Why it matters:

Students should understand that security is part of normal software development. It should not be treated only as a separate subject studied later in the programme.

Source: [CNCF and SlashData, State of Cloud Native Development Q1 2026](#)

Accessed: September 2026

4.7 Python Has Become More Important with AI and Data

Python is already included in the R22 curriculum through **CS206ES – Python Programming Laboratory**. Its importance has increased further because Python is widely used in AI, machine learning, data science and backend development.

The 2025 Stack Overflow Developer Survey reported a **7 percentage-point increase in Python adoption from 2024 to 2025**, connecting its growth with AI, data science and backend development.

Affected courses:

CS206ES – Python Programming Laboratory
CS601PC – Machine Learning
CS603PC – Artificial Intelligence

What should change:

Python practical work can be connected more directly with data processing, APIs and AI/ML applications.

Why it matters:

Students can use the same programming language across programming, data and AI-related work instead of treating each area as completely separate.

4.8 The Skills Expected from Graduates Are Changing Quickly

One of the biggest changes is not related to a particular programming language or technology. It is the speed at which required skills are changing.

The World Economic Forum estimates that **39% of workers' existing skill sets are expected to be transformed or become outdated between 2025 and 2030**. It also identifies AI and

big data, networks and cybersecurity, and technological literacy among the fastest-growing skills.

Affected courses:

CS405PC – Software Engineering

CS503PC – DevOps

CS601PC – Machine Learning

CS603PC – Artificial Intelligence

What should change:

Along with technical fundamentals, students should get regular opportunities to learn unfamiliar tools, work with new technologies and adapt their existing knowledge to new situations.

For example, instead of focusing on one fixed tool, students can be given projects where they have to:

Understand the problem → Learn the required tool → Apply the fundamentals → Test the solution → Adapt

Why it matters:

A particular framework, platform or AI tool taught today may not be equally important in 2030. However, the ability to learn, evaluate and adapt to new technologies will remain useful.

4.9 Main Finding

These eight changes show that the biggest gap in the R22 CSE curriculum is **not the disappearance of Computer Science fundamentals**.

The major change is **how those fundamentals are used in the current technology environment**.

The basic R22 foundation can be represented as:

Programming → Algorithms → DBMS → OS → Networks → AI

The 2026 environment adds practical layers around the same foundation:

Programming + AI

Algorithms + AI-assisted solutions

DBMS + APIs

OS + Cloud

Networks + Security

AI + LLMs/Agents

Therefore, the main finding from this review is: **“The foundation should remain stable, but the practical layer should be updated continuously.”**

SECTION 5: Curriculum vs Industry

A CSE graduate is expected to do more than just understand programming concepts. From the changes identified in Section 4, I found that industry increasingly expects graduates to combine programming fundamentals with software development practices, cloud technologies, testing, cybersecurity, AI tools and teamwork.

The R22 curriculum already provides many of these foundations through subjects such as Programming, Data Structures, Object-Oriented Programming, DBMS, Software Engineering, Networks, DevOps, Machine Learning and Artificial Intelligence. The main difference is that industry expects these skills to be used together in practical development environments.

Therefore, the purpose of this comparison is not to say that R22 is outdated or unsuitable. Instead, it is to identify where the existing foundation agrees with industry expectations and where the curriculum needs stronger practical integration.

5.1 What Industry Expects from a CSE Graduate

Based on the job descriptions reviewed, a CSE graduate should be able to:

- Write and understand programs using a suitable programming language.
- Apply data structures and algorithms to solve problems.
- Design and develop software applications.
- Work with databases and SQL.
- Understand APIs and application integration.
- Use Git and collaborative development practices.
- Test, debug and improve software.
- Understand basic cloud and deployment workflows.
- Work with CI/CD and modern software development practices.
- Understand basic cybersecurity and secure development.
- Use AI-assisted development tools responsibly and verify their output.
- Understand the basics of AI/ML and their practical applications.
- Work effectively in a team and communicate technical work.
- Learn new tools and technologies as industry requirements change.

These expectations are consistent with the changes identified in Section 4, particularly the growth of AI-assisted development, cloud-native development, cybersecurity and automated software workflows.

The important observation is that these are not completely separate skills. In an actual software project, a developer may need to use programming, databases, APIs, Git, testing, security and deployment as parts of the same development process.

5.2 Real Job Description 1 – Microsoft

Role: Software Engineer – Recent Graduate

Company: Microsoft

Microsoft's recent-graduate career information describes Software Engineers as professionals who design, develop and maintain software applications. The role also involves software engineering practices, collaboration, troubleshooting, continuous learning and adapting to new technologies.

Official job source:

[Microsoft – Recent Graduate Software Engineer](#)

Agreement with R22

R22 provides a strong foundation for this type of role through:

- **CS103ES – Programming for Problem Solving**
- **CS302PC – Data Structures**
- **CS305PC – Object Oriented Programming through Java**
- **CS404PC – Database Management Systems**
- **CS405PC – Software Engineering**
- **CS503PC – DevOps**
- Project and internship/skill-development components

These subjects cover many of the fundamental concepts expected from an entry-level software engineer.

Difference from R22

The main gap is not the absence of software engineering fundamentals. The gap is the **integration of these skills**.

In an industry environment, a developer may be expected to write code, use version control, review code, test the application, fix bugs, work with other developers and contribute to deployment and maintenance.

R22 teaches many of these areas, but they are distributed across different subjects and laboratories. More end-to-end development using Git, APIs, testing, debugging, deployment and team-based workflows would make the curriculum closer to the actual software development environment.

The lesson from this job description is therefore not that another programming subject is required. Instead, existing subjects can be connected through practical projects that require students to use several skills together.

5.3 Real Job Description 2 – Deloitte

Role: Full Stack Developer

Company: Deloitte Shared Services India LLP

Location: Hyderabad

A Deloitte Full Stack Developer role involves developing and maintaining applications and working across areas such as backend services, SQL, user interfaces, APIs and application frameworks.

Official job source:

[Deloitte – Full Stack Developer, Hyderabad](#)

Agreement with R22

There is a clear connection with the following R22 components:

- **CS305PC – Object Oriented Programming through Java**
- **CS404PC – Database Management Systems**
- **CS405PC – Software Engineering**
- **CS409PC – Node JS / React JS / Django**
- **CS503PC – DevOps**
- **CS506PC – UI Design – Flutter**
- Professional electives related to Full Stack Development and Cloud Computing

Therefore, R22 already contains several technologies and concepts that are relevant to full-stack development.

Difference from R22

The main difference is the **depth of integration**.

The syllabus provides web technologies, DBMS, software engineering and DevOps through different courses. However, a full-stack developer is expected to connect these areas while building an actual application.

A typical full-stack workflow can be represented as:

Frontend → Backend → API → Database → Testing → Deployment

For example, a student may know React from one course and SQL from another course, but the stronger industry-oriented experience comes from building an application where the React frontend communicates with a backend API, stores information in a database, is tested and is finally deployed.

This supports the change identified in Section 4 that software development is becoming increasingly integrated and cloud-oriented.

The curriculum therefore does not necessarily need to add more individual technologies. Instead, existing technologies should be brought together through larger practical assignments and projects.

5.4 Real Job Description 3 – Wipro

Role: Graduate Engineer Trainee

Company: Wipro

Location: Gurugram

A Wipro Graduate Engineer Trainee role includes activities such as preparing test data, performing database checks, assisting with unit testing, analysing test results and documenting technical work.

Wipro's graduate hiring programmes also provide newly graduated engineers with opportunities to work on projects, receive industry-oriented training and gain exposure to real-world client environments.

Official job source:

[Wipro – Graduate Engineer Trainee](#)

Agreement with R22

This connects with:

- **CS405PC – Software Engineering**
- **CS404PC – Database Management Systems**
- **CS406PC – Operating Systems Lab**
- **CS407PC – Database Management Systems Lab**
- **CS503PC – DevOps**
- Project and internship/skill-development components

R22 therefore provides the basic academic knowledge required for software testing, databases and software development.

Difference from R22

The important point from this job description is that even an entry-level role can involve practical activities such as testing, database verification, documentation and analysing results.

R22 includes these concepts, but students could receive more practice through realistic development tasks rather than treating them mainly as separate academic subjects or laboratory exercises.

For example, instead of only performing an isolated database experiment and a separate programming experiment, a practical assignment could require students to:

Develop → Connect Database → Test → Debug → Document → Submit through Git

This would allow students to understand how individual academic concepts are used together in an actual software workflow.

5.5 Where the Degree Agrees and Disagrees with Industry

The comparison shows that R22 is not fundamentally disconnected from industry. Most of the important foundations required by software companies are already present in the curriculum.

For example:

- Programming is covered.
- Data Structures and Algorithms are covered.
- Object-Oriented Programming is covered.
- DBMS and SQL are covered.
- Software Engineering is covered.
- Web development is covered.
- DevOps is covered.
- AI and Machine Learning are covered.
- Cybersecurity is included.
- Projects and internship/skill-development components provide opportunities for application.

Therefore, the main issue is not simply **what subjects are present**.

The stronger issue is **how these subjects are connected and applied**.

For example, R22 teaches Programming, DBMS, Software Engineering and DevOps, but a modern software developer is expected to combine them into one workflow. Similarly, R22

contains AI and Machine Learning, but the current technology environment also includes AI-assisted development, LLM-based applications and AI-supported software workflows.

This creates a difference between **subject-level learning** and **industry-level application**.

A student may complete individual subjects successfully but still need more experience in taking a problem from requirements to a working, tested and deployed application.

Therefore, I would not recommend replacing the Computer Science foundation with only new technologies.

My finding is:

R22 provides many of the foundations industry expects, but the curriculum needs to give students more opportunities to combine those foundations into modern, practical development workflows.

This also supports the main conclusion from Section 4:

The foundation should remain stable, but the practical layer should be updated continuously.

5.6 Final Finding from the Industry Comparison

After comparing the R22 curriculum with the three job descriptions, I found that there is a considerable amount of agreement between the degree curriculum and industry expectations.

The major difference is the **level of practical integration**.

Industry does not normally use Programming, DBMS, Software Engineering, DevOps, AI and Security as isolated topics. These skills are combined while solving real problems and developing software.

Therefore, the curriculum should provide more opportunities for students to work through complete development cycles:

Problem → Requirements → Design → Code → Database → API → Git → Testing → Security → Deployment → Documentation

This does not require removing the Computer Science foundation. Instead, it requires better use of curriculum time and stronger connections between existing courses, laboratories, projects and internships.

SECTION 6: My 2030 Curriculum – From Learning Subjects to Building Skills

The review in Sections 3, 4 and 5 leads to one main conclusion:

R22 has a strong Computer Science foundation, but students need more opportunities to apply that foundation to modern software development.

My concern with newer curricula such as R25 is not the inclusion of modern technologies. Areas such as Generative AI, Prompt Engineering, Deep Learning, Cloud and Cybersecurity are important. The concern is that continuously adding new topics can increase the workload of a four-year degree.

If every new technology becomes another subject, students may cover more topics but have less time to understand and apply them deeply.

Therefore, my approach is:

Do not keep adding subjects. Rebalance the learning.

Instead of: **New Technology → New Subject → Exam**

I propose: **Foundation → Practice → Integration → Application → New Technology**

Foundation → Practice → Integration → Application → New Technology

This keeps the degree manageable while improving its practical relevance.

6.1 The Main Design

Current Challenge	My 2030 Direction
Too many separate topics	Reduce fragmentation
Subjects have limited connection	Integrate practical work
New technologies keep being added	Teach them through existing foundations
AI is changing programming	AI-assisted development + verification
Development involves more than coding	Git + APIs + Testing + Deployment
Cloud is becoming important	Connect Networks + DevOps + Cloud
Security is often treated separately	Apply security throughout development
Projects mainly come at the end	Build progressively from Year 1

6.2 My 2030 Student Journey

I would design the degree as a four-stage skill journey rather than eight semesters of separate subjects.

Year	Focus	Student Outcome
Year 1	Learn & Build	Build small software
Year 2	Connect	Develop complete applications
Year 3	Develop & Deploy	Work with Cloud, DevOps and AI
Year 4	Engineer & Deliver	Build and deliver complete solutions

6.3 What Changes Across the Four Years?

Year 1 – Build the Foundation

Focus on: **Programming + Problem Solving + Git + Linux + Debugging + Documentation**

Students begin building small applications instead of only completing isolated laboratory exercises.

Goal: “I can write and understand a program.”

Year 2 – Connect the Foundation

Connect: **Data Structures + OOP + DBMS + Web Development + APIs + Software Engineering**

Practical work follows: **Frontend → Backend → API → Database → Testing → Git**

Goal: “I can develop a complete application.”

Year 3 – Develop, Deploy and Use AI

Existing subjects become entry points for modern technologies:

- **Networks → Cloud**
- **DevOps → CI/CD + Containers**
- **ML → Deployment + Monitoring**

- **AI → LLMs + Evaluation + Agents**
- **Programming → AI-assisted coding + Verification**

Goal: “I can develop, deploy and work with modern technologies.”

Year 4 – Engineer and Deliver

Bring together: **Security + Testing + Deployment + Monitoring + Documentation + Project**

The final project follows:

Problem → Design → Code → Data → API → Test → Secure → Git → Deploy → Document

Goal: “I can take a problem and deliver an engineered solution.”

6.4 Credit Rebalancing Under the 160-Credit Constraint

The redesigned degree remains at **160 credits**. I am redesigning **32 credits**, which represents exactly **20% of the programme**:

$$32 / 160 \times 100 = 20\%$$

This remains within the proposed 20–25% redesign range.

Importantly, I am **not adding 32 new credits**. The 32 redesigned credits consist of:

- **14 credits reallocated** from selected existing components
- **18 existing core credits substantially updated** through revised learning outcomes and practical application

Semester-wise Credit Rebalancing:

Semester	Reduced / Reworked Source	Credits Released	2030 Component Added / Reallocated	Credits	Prerequisite
I-I	Engineering Chemistry: 4 → 3	1	Computing Practice	+1	None
I-II	Applied Physics: 4 → 3	1	Software Building Lab	+3	Programming / Computing Practice

Semester	Reduced / Reworked Source	Credits Released	2030 Component Added / Reallocated	Credits	Prerequisite
I-II	Engineering Workshop: 2.5 → 1.5	1	—	—	—
I-II	Engineering Graphics: 3 → 2	1	—	—	—
II-I	Computer Oriented Statistical Methods: 4 → 3	1	Integrated Programming Practice	+1	Programming + Data Structures
II-II	BEFA: 3 → 2	1	Application Integration Lab	+2	OOP + DBMS
II-II	English for Skill Enhancement: 2 → 1	1	—	—	—
III-I	Advanced English Communication Skills Lab: 1 → 0*	1	Cloud & Deployment Practice	+2	Networks + DevOps
III-I	DevOps: 3 → 2	1	—	—	—
III-II	Big Data/Spark skill component: 2 → 1	1	AI Engineering Practice	+2	Machine Learning + AI
III-II	OE-I: 3 → 2	1	—	—	—
IV-I	OE-II: 3 → 2	1	Production Engineering Practice	+2	Software Engineering + DevOps + CNS
IV-I	Project Stage-I: 3 → 2	1	—	—	—
IV-II	Organizational Behavior: 3 → 2	1	Engineering Review & Delivery	+1	Project Stage-I

Total credits released = 14

Total credits reallocated = 14

The communication outcomes are not removed completely. They are shifted into project documentation, presentations, reviews and engineering practice.

Credit Balance

Item	Credits
Original R22 programme	160
Credits reduced/reallocated	−14
New practical/integrated components	+14
Final programme	160

Therefore, every new practical component is paid for by a named reduction or reallocation within the existing programme. **No additional credits are added to the degree.**

The remaining **18 redesigned credits** are not new credits. They are existing core credits whose learning outcomes and practical application are updated, particularly in OOP, DBMS, Networks, Software Engineering, DevOps, ML and AI.

What the 14 Reallocated Credits Create

The released credits are redirected towards:

Computing Practice → Software Building → Application Integration → Cloud & Deployment → AI Engineering → Production Practice

The redesign therefore does not create a long list of new subjects. Instead, it creates connected practical experiences around the subjects students already study.

6.5 What Happens to the 18 Updated Credits?

The other 18 credits are improved without creating additional credits.

Existing Area	2030 Update
Programming	AI-assisted development and verification
OOP	Git, debugging, testing and code review
DBMS	API, backend and database integration
OS	Processes, threads, containers and cloud concepts

Existing Area	2030 Update
Networks	DNS, HTTPS, APIs and cloud networking
Software Engineering	Git, issue tracking, testing and CI/CD
DevOps	Containers, deployment and monitoring
ML	Deployment, evaluation and monitoring
AI	LLMs, evaluation, limitations and agents

So the redesign is:

14 credits reallocated + 18 credits updated = 32 redesigned credits

rather than: **32 completely new credits**

6.6 AI as a Development Skill

AI should not become simply:

AI → New Syllabus → Exam → Certificate

Instead, it should appear throughout the learning process.

Examples include:

- **Programming:** AI-assisted coding + verification
- **DBMS:** AI-generated SQL + testing
- **Software Engineering:** AI-assisted debugging and documentation
- **AI/ML:** LLMs, evaluation, limitations and agents
- **Project:** Document AI usage and verify outputs

AI becomes a development skill, not just another examination topic.

6.7 How My Approach Differs from R25

I agree with R25's movement towards newer technologies. My difference is not about rejecting modern subjects. It is about how much of the curriculum should become a separate course.

R25 introduces areas such as Deep Learning, Prompt Engineering, NLP, Cybersecurity and other newer technology-focused learning. My 2030 design takes a more integrated approach.

R25-style Direction:

New Technology → New Topic/Course → Separate Learning

My 2030 Direction:

Existing Foundation → New Technology → Integrated Practice → Application

Why I Choose This Approach

R25 shows that the curriculum needs to respond to technology changes. I agree with that direction.

However, my concern is:

If every emerging technology becomes another standalone subject, curriculum coverage can grow faster than students' time to practise and connect what they learn.

My proposal therefore uses the limited **160-credit space** differently:

Keep the foundation → Reduce selected low-priority/repetitive allocation → Integrate practical skills → Apply emerging technologies through the foundation.

This is why my redesign focuses on **20% of the programme rather than restructuring the entire degree.**

6.8 What a 2030 Graduate Should Be Able to Do

A graduate should be able to take a problem through:

Problem → Design → Programming → Data → API → Testing → Security → Git → Deployment → Documentation

The goal is not to become an expert in every technology that exists in 2030.

The goal is to graduate with the ability to:

Learn → Build → Integrate → Test → Secure → Deploy → Adapt

When a new technology appears, the student should be able to connect it with the Computer Science foundation they already have.

My 2030 curriculum therefore does not try to predict every technology of 2030. It prepares students to learn, apply and adapt to those technologies without losing the fundamentals.

SECTION 7: What Would Make Me Wrong?

My conclusion is that we do not need to keep replacing the Computer Science foundation whenever technology changes. Instead, students need more opportunities to apply that foundation to new technologies and real-world development practices.

However, while making this proposal, I also asked myself an important question:

What if I am wrong?

A good curriculum proposal should not only explain why it may work. It should also identify what evidence could prove it wrong.

7A. What Evidence Would Disprove My Claim?

I would reconsider my proposal if real evidence showed that teaching emerging technologies as separate subjects consistently produces better outcomes than integrating them into existing subjects.

For example, my approach would need to change if:

- Employers consistently preferred graduates with separate courses for each emerging technology.
- Students performed better when AI, Cloud Computing or Deep Learning were taught as separate subjects.
- Reducing or integrating existing subjects resulted in important Computer Science fundamentals being lost.
- Students from more integrated curricula consistently achieved weaker project, internship or placement results.

This is important because my proposal should ultimately be judged by **results, not just by my opinion.**

7B. What I Believed at the Start — and What Changed?

When I started this review, I had a simple assumption:

“If technology keeps changing, the curriculum should keep adding new subjects.”

After examining my R22 curriculum, current technology changes, industry requirements and R25, I started seeing the problem differently.

A four-year degree has limited time and 160 credits. If we continuously add subjects whenever a new technology becomes important, students may get exposure to more topics but have less time to practise, build projects and connect different areas of Computer Science.

My thinking therefore changed from:

Add more subjects to stay current.

to:

Use the existing foundation better and integrate new technology through practice.

This change in thinking is what led to my proposed 2030 curriculum.

7C. What I Could Not Verify

There are some things that I could not honestly claim to know.

I could not verify exactly what CSE employers will expect from graduates in 2030. The job descriptions, industry reports and technology trends reviewed in this study show current and near-term requirements, but they cannot predict every future requirement.

I also could not prove that **20% is the ideal amount of curriculum to redesign**. I selected 20% because it is within the challenge's 20–25% limit and allows meaningful changes without rebuilding the entire 160-credit degree.

Therefore, the 20% figure should be treated as a **practical design choice**, not as a claim that exactly 20% is scientifically necessary.

These limitations are important because they separate what I have supported with evidence from what remains a proposal or assumption.

Final Reflection

The most important thing I learned from this exercise is that a curriculum review should not start with:

“What new subjects can we add?”

It should start with:

“What is already working? What has changed? What is missing? What can we improve without making the degree unnecessarily bigger?”

My proposal may not be the perfect curriculum for 2030. My goal is not to predict the future.

My goal is to prepare students to adapt to it.

Verification date: 12 September 2026.

SECTION 8: AI Verification – What I Tested and Learned

As part of my curriculum review, I also tested AI with some common Computer Science tasks. My purpose was not only to see whether AI could generate answers, but also to check whether those answers were correct. I selected five areas from the R22 CSE curriculum: Programming, DBMS, Software Engineering, Machine Learning and Artificial Intelligence.

This helped me understand an important point: **AI can make learning and development faster, but students still need Computer Science knowledge to check, test and verify AI-generated output.**

AI model used: ChatGpt – GPT-5.6 Luna

Testing period: September 2026

8.1 Programming

Prompt I tested:

“Write a Python program to find the second-largest number in a list, including duplicate values.”

What AI gave:

AI generated a Python program along with an explanation of the logic.

What I checked:

I tested the program with different inputs, including lists containing duplicate values and different number arrangements.

My finding:

AI can help students write programs quickly, but the student still needs to understand the logic and test the program with different inputs. A generated program should not be accepted without checking its behaviour.

8.2 DBMS

Prompt I tested:

“What happens when a DELETE statement is executed without a WHERE clause? Does it delete only one record or all records in the target table? Explain with a simple example.”

What AI gave:

The AI stated that a DELETE statement without a WHERE clause deletes only one record.

What I checked:

I verified the statement against the SQL/DBMS material and tested the behavior.

My finding:

The answer was incorrect. A DELETE statement without a WHERE condition can delete all rows in the target table.

8.3 Software Engineering

Prompt I tested:

“Create test cases for a student login system.”

What AI gave:

AI generated positive and negative test cases, including valid login, incorrect password and empty fields.

What I checked:

I checked whether the important cases were covered and whether any situations were missing.

My finding:

AI can help generate test cases and save time during testing. However, human review is still necessary because AI may miss application-specific or unusual cases.

8.4 Machine Learning

Prompt I tested:

“Explain the steps of building a classification model from data collection to deployment.”

What AI gave:

The response described a workflow such as:

Data Collection → Data Preparation → Preprocessing → Training → Evaluation → Deployment

What I checked:

I compared the workflow with the Machine Learning process I had already studied.

My finding:

AI is useful for explaining ML workflows and concepts. However, understanding the steps is different from actually building a model. Students still need practical experience with data, model training, evaluation and deployment.

8.5 Artificial Intelligence

Prompt I tested:

“Explain the difference between an LLM-based assistant and an AI agent.”

What AI gave:

AI explained that an LLM-based assistant mainly generates responses, while an AI agent can use tools, interact with external systems and perform multi-step tasks toward a goal.

What I checked:

I compared the explanation with current information about AI systems and the changing role of AI in software development.

My finding:

AI concepts are developing very quickly. Therefore, students need to learn not only how these systems work, but also their limitations, reliability, responsible use and the need to verify their outputs.

8.6 One AI Answer I Found Was Wrong

I also tested whether AI would always provide a technically correct answer.

Prompt I tested:

“Does DELETE FROM Students delete only one record?”

The AI response said that it would delete only one record. I checked this because SQL behaviour can be verified directly from database concepts and documentation.

The answer was incorrect. A DELETE statement without a WHERE condition can delete **all rows** from the table.

What I learned:

AI can produce an answer that sounds confident even when the answer is technically wrong. Therefore, students should not blindly trust AI-generated answers. They should test the output, understand the underlying concept and verify important technical information using reliable documentation or other trusted sources.

8.7 What I Learned from the AI Test

The main lesson from my testing was:

AI can help me learn and build faster, but I cannot blindly trust its output.

This also influenced my curriculum proposal. I do not think AI should be treated only as another separate subject that is added whenever technology changes. Students should learn

how to **use AI, understand its output, test it, identify mistakes and verify the result** while continuing to build strong Computer Science fundamentals.

This connects directly with my proposed 2030 curriculum approach:

Foundation → AI-Assisted Work → Testing → Verification → Application

My testing showed that AI can be useful for programming, explanations, debugging and generating examples, but its output cannot be accepted automatically. The important skill is not only using AI, but also checking, testing and verifying its output using reliable technical sources.

This supports my 2030 curriculum proposal that AI-assisted development should be taught together with verification and testing rather than treated simply as a separate tool-based subject.

Sources:

1. [JNTUH R22 B.Tech CSE Syllabus](#) – Used to verify the subjects and course content tested in this section.
2. [Stack Overflow Developer Survey 2025](#) – Used to understand current developer use and concerns regarding AI tools.
3. [IBM SQL DELETE Documentation](#) – Used to verify the behavior of DELETE statements and the effect of omitting a WHERE clause.
4. [Russell & Norvig -Artificial Intelligence](#) – A Modern Approach Reference for AI concepts covered during verification.
5. [Marsland -Machine Learning](#) – An Algorithmic Perspective-Reference for machine-learning concepts used during verification.